Instituto Tecnológico de Costa Rica



Primer Proyecto Programado:
Sistema de Soda "SodaJava"

Grupo 6:
Sebastian Rodriguez Vargas – Carné: 2026800908
Cristian David Bolaños Acuña – Carné: 2025064251
Sara Saborío Quesada – Carné: 2026012410

Profesor:
Ing. Emmanuel Ramírez Segura

Fecha de entrega:
27/10/26

II Semestre, 2026

Índice

1. Objetivos.....3
2. Desarrollo..... 4
2.1 Diagrama de clases UML.....4
2.2 Casos de Uso de éxito y fallido del Cajero.....5
2.2.1 Caso de Uso UC-01: Procesar pago de pedido.....5
2.3 Diagrama de Casos de Uso UML del escenario de éxito y escenario fallido.....6
3. Bibliografía..... 6

1. Objetivos

Objetivo General:

Automatizar procesos operativos y de gestión de la soda "SodaJava" mediante el desarrollo de un programa orientado a objetos en Java, utilizando el paradigma de Programación Orientada a Objetos (POO), para controlar de manera eficiente cosas como la toma de pedidos, la administración de inventarios de porciones, la facturación con cálculo de IVA y también el monitoreo en tiempo real del estado de las órdenes.

Objetivos Específicos:

- **Diseñar la arquitectura del sistema** mediante un Diagrama de Clases en UML y casos de uso detallados, estructurando las entidades del problema (usuarios, roles, clientes, productos y facturas) bajo principios de la Programación Orientada a Objetos y también el uso de estructuras de datos adecuadas.
- **Implementar los módulos funcionales y de control de acceso** en Java (con interfaz gráfica en NetBeans), aplicando validaciones para el descuento de inventario por porciones, el manejo de estados de facturación y órdenes, y la separación de roles entre personal administrativo, cajeros y cocineros.

2. Desarrollo

2.1 Diagrama de clases UML

El diseño de arquitectura de software de SodaJava se fundamenta en los principios de la Programación Orientada a Objetos (osea POO). El siguiente diagrama UML ilustra las clases principales del sistema, como Administrador, Cajero, Cocinero, Cliente, Producto y Factura junto con sus respectivos atributos, sus métodos y las estructuras de datos seleccionadas para gestionar la cardinalidad y el flujo de información del negocio:

2.2 Casos de Uso de éxito y fallido del Cajero

2.2.1 Caso de Uso UC-01: Procesar pago de pedido

- **Descripción:** Permite al cajero procesar el pago de un pedido existente en estado pendiente, validando el inventario de las porciones y generando también la factura final.
- **El actor principal:** El cajero
- **Precondiciones:**
    - El Cajero debe de haber iniciado sesión en el sistema.
    - Debe existir una factura previamente creada en estado “pendiente” con las líneas de productos correspondientes.

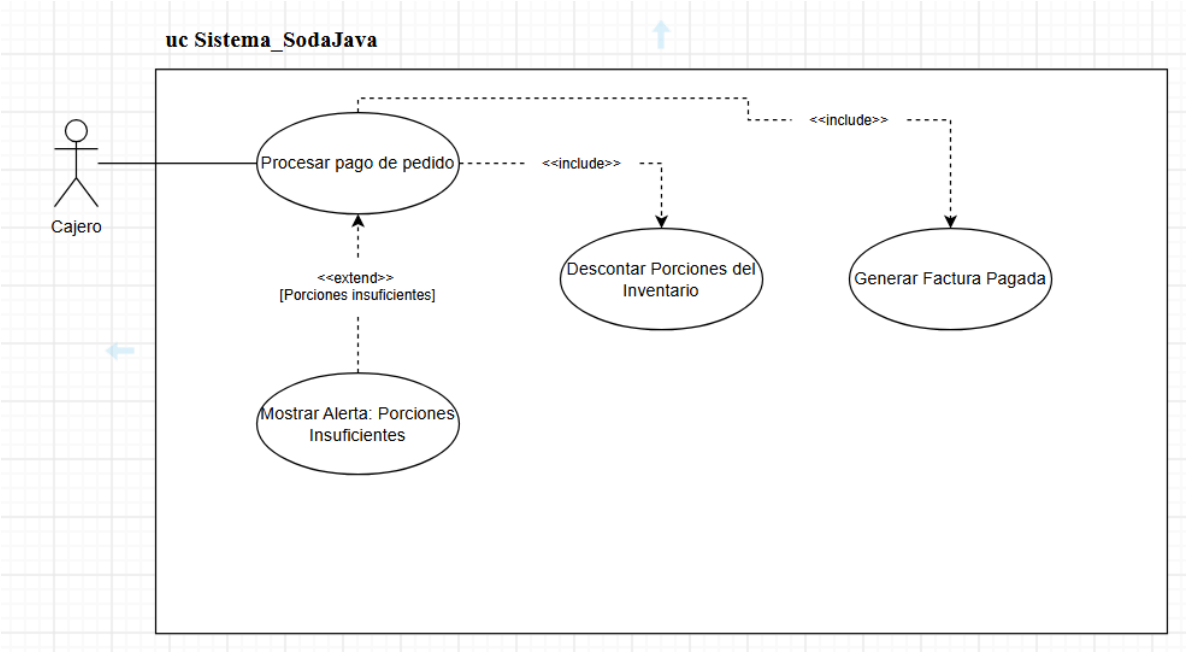
- **1) Escenario de éxito (El flujo principal)**
    a. El cajero selecciona del sistema la factura que esté pendiente asociada al cliente.
    b. El cajero activa la opción de “pagar factura”.
    c. El sistema verifica que la factura se encuentre en estado pendiente y procede a revisar las porciones disponibles del menú.

d. Validación exitosa: el sistema comprueba que si existen las porciones suficientes en nuestro inventario para cada una de las cosas que se pedían en la factura.
e. El sistema descuenta automáticamente del inventario esas porciones vendidas.
f. El sistema actualiza el estado de factura de “pendiente” a “pagada”.
g. El sistema almacena la factura de forma permanente en el historial del sistema y cliente.
h. El sistema envía la orden a la pantalla de salida con el estado de “en preparación”.
i. El sistema muestra en la pantalla el reporte de la factura para el cajero.

- **2) Escenario fallido o excepción (El flujo alternativo por una falta de porciones)**
    - Este flujo se activa en el paso 4 del escenario principal pero cuando el inventario no es suficiente.
    a. El cajero selecciona la factura pendiente y presiona la opción “pagar factura”.
    b. Validación fallida: el sistema revisa inventario y detecta que uno o varios productos no cuentan con la cantidad que se solicita de porciones.
    c. El sistema bloquea e impide que el pago sea procesado, evitando que de ninguna forma existan cantidades negativas en el inventario.
    d. El sistema muestra una alerta o mensaje de advertencia en pantalla al cajero que le indica que faltan porciones de productos.
    e. La factura mantiene obligatoriamente el estado de “pendiente” y las porciones del menú permanecen intactas.
    f. El cajero procede a modificar o eliminar las líneas del pedido afectado según lo requiera el cliente.

2.3 Diagrama de Casos de Uso UML del escenario de éxito y escenario fallido

Aquí se presenta el Diagrama de Casos de Uso UML que modela el comportamiento del sistema *SodaJava* ante las interacciones del Cajero. Este esquema ilustra gráficamente el flujo principal para el procesamiento de pagos exitosos, el cual, incluye de manera obligatoria la reducción automática de porciones en el inventario y la emisión de la factura; Y también el flujo alternativo de excepción que se activa y bloquea la transacción cuando se detecta una escasez de porciones de productos:



3. Bibliografía

Oracle. (s. f.). Object-oriented programming concepts. https://docs.oracle.com/javase/tutorial/java/concepts/

Deitel, H. M., & Deitel, P. J. (2017). *Java: Cómo programar* (10.ª ed.). Pearson Educación. Enlace de referencia

Oracle. (s. f.). Object-oriented programming concepts. Oracle Java Tutorials. https://docs.oracle.com/javase/tutorial/java/concepts/

Vega, M. (2010). *Casos de uso UML* [Apuntes de clase, Departamento de Lenguajes y Sistemas Informáticos]. Universidad de Granada. https://lsi2.ugr.es/~mvega/docis/casos%20de%20uso.pdf